

# 学习养成计划软件 需求规格说明书

## 1. 业务需求概述

本软件旨在解决用户在学习过程中缺乏规划、难以坚持、复习遗忘的问题。通过提供任务管理、智能提醒、打卡正反馈及数据可视化分析，帮助用户建立良好的学习习惯，形成“计划-执行-复习-总结”的闭环。

## 2. 需求模型：用户故事

使用敏捷开发中的 User Story 来描述各类业务需求，格式为：作为<角色>，我想要<功能>，以便于<商业价值/目的>。

### 2.1 学习任务模块

- **US1.1:** 作为用户，我想要自定义添加学习任务，以便于规划我个人的学习内容。
- **US1.2:** 作为用户，我想要从系统推荐任务库中一键添加任务，以便于快速开始常见的学习计划（如：背单词 30 天、考研数学打卡等）。
- **US1.3:** 作为用户，我想要为任务设置主题（分类）、完成时间和优先级，以便于我对任务进行分类和排序。

### 2.2 任务提醒模块

- **US2.1:** 作为用户，我想要系统根据任务的完成时间发送提醒，以便于我不会遗漏截止日期。
- **US2.2:** 作为用户，我想要高优先级任务比低优先级任务获得更频繁的提醒，以便于我能集中精力处理重要紧急的任务。
- **US2.3:** 作为用户，我想要在开启“复习提醒”后，系统能按记忆曲线（如 1 天、3 天、7 天）提醒我复习，以便于我巩固学习成果。

### 2.3 打卡任务模块

- **US3.1:** 作为用户，我想要在完成任务后进行打卡操作，以便于记录我的学习轨迹。
- **US3.2:** 作为用户，我想要系统自动生成打卡记录并累计连续打卡天数，以便于获得成就感并维持学习动力。

### 2.4 数据展示与任务分析模块

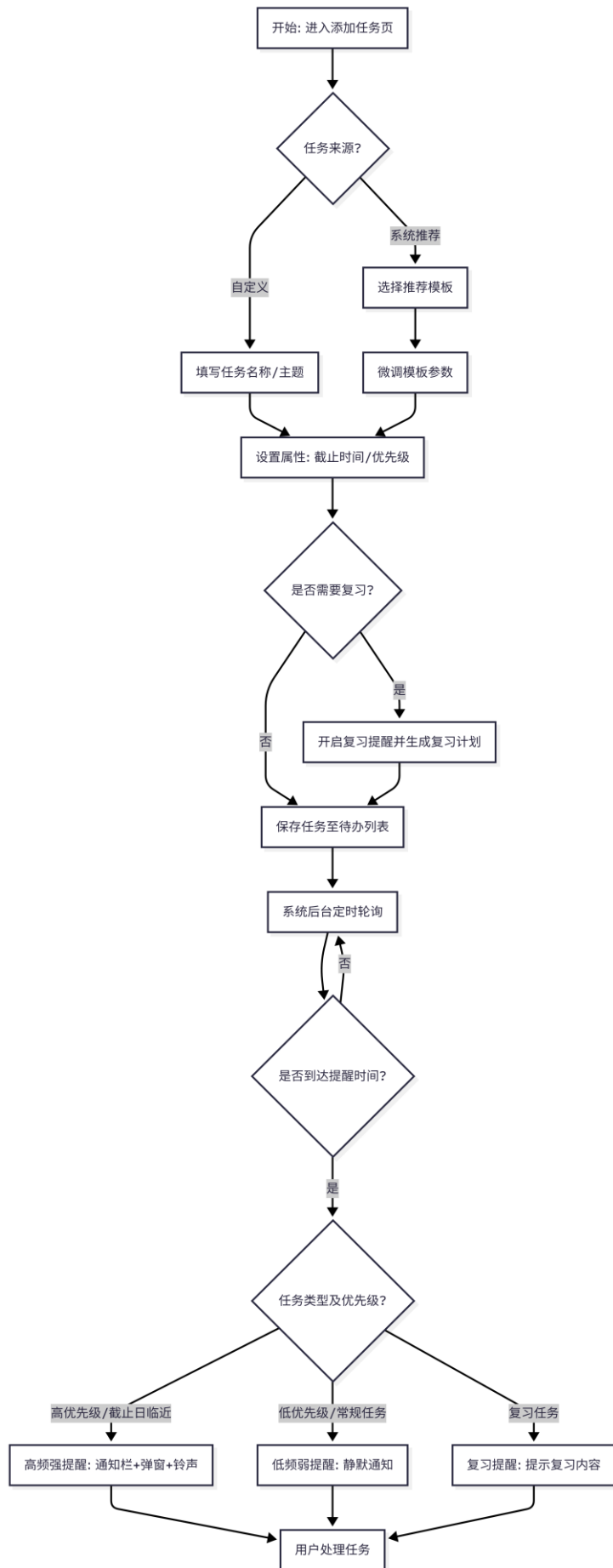
- **US4.1:** 作为用户，我想要通过进度条查看当前任务的完成进度，以便于直观了解剩余工作量。

- **US4.2:** 作为用户，我想要通过打卡日历查看历史打卡记录，以便于回顾我的坚持过程。
  - **US4.3:** 作为用户，我想要查看每日任务添加数和完成率的统计表，以便于分析我的学习效率并调整计划。
- 

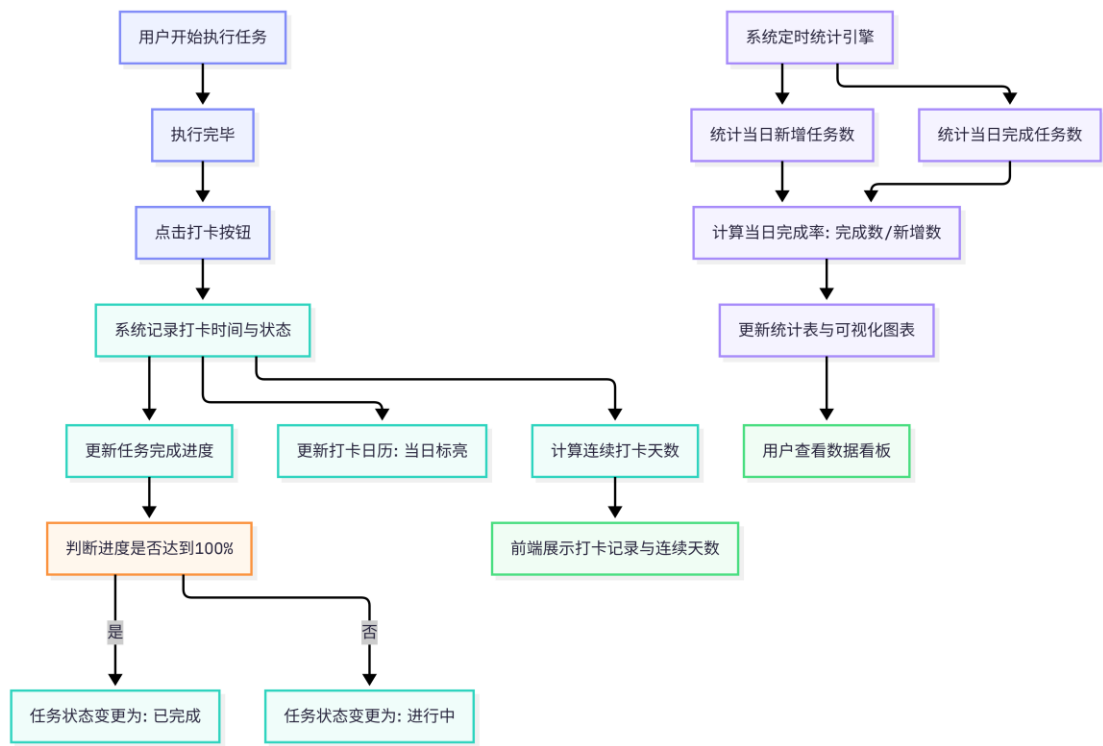
### 3. 需求模型：业务流程图

以下是核心业务流程的模型图，使用 Mermaid 语法绘制（支持 Mermaid 的 Markdown 阅读器可直接渲染图形）：

#### 3.1 任务创建与提醒业务流程

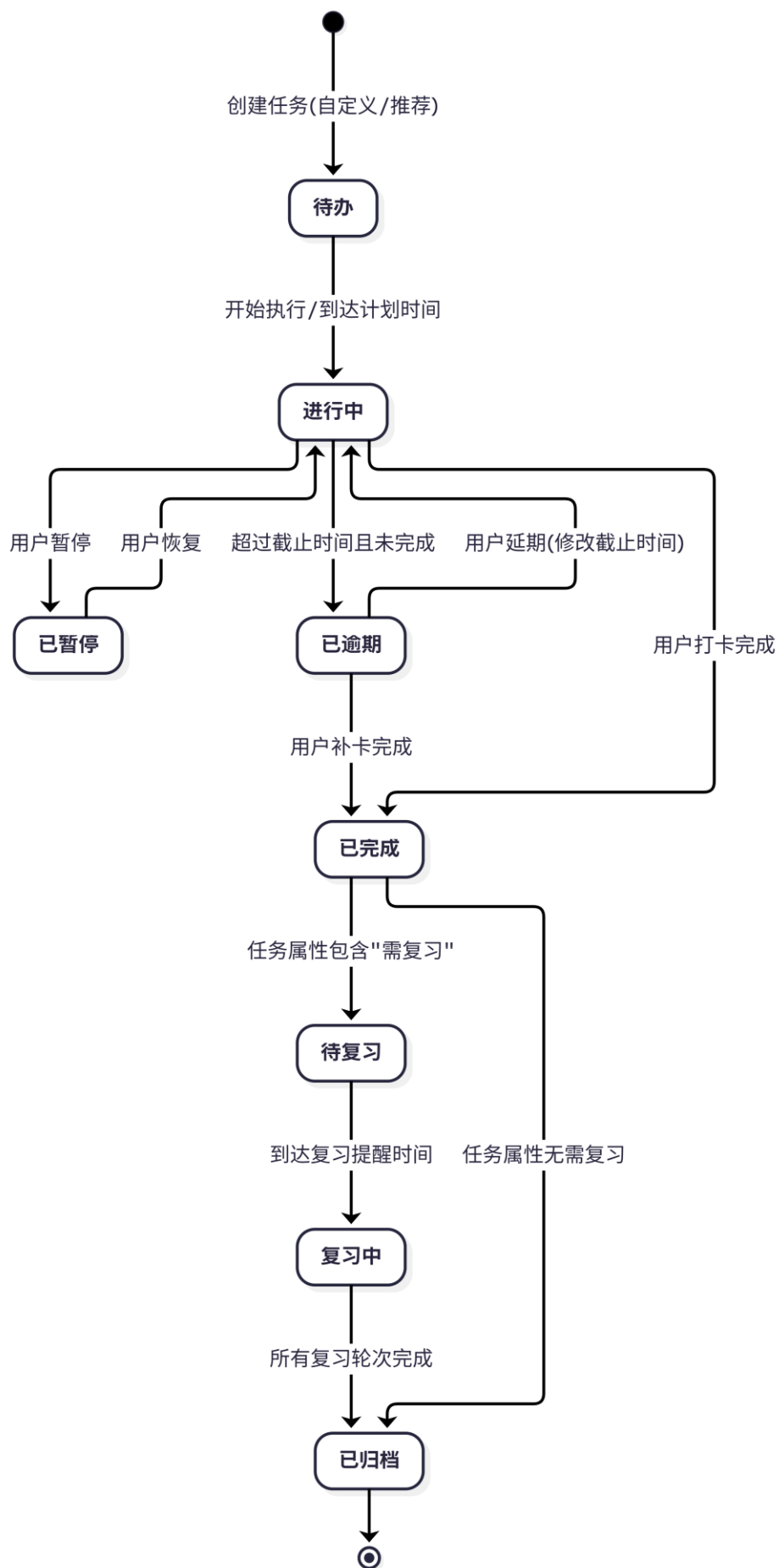


3.2 任务打卡与数据分析业务流程



4. 需求模型：任务状态机图

为了更精确地说明“任务”这一核心对象在系统中的生命周期，使用状态机图进行建模：



## 5. 核心业务规则说明

为了确保需求模型的可执行性，对流程和状态图中涉及的模糊概念进行规则约束：

### 1. 提醒频率规则：

- **高优先级：**截止日前 3 天每天提醒 2 次，截止日当天每 2 小时提醒 1 次。
- **中优先级：**截止日前 1 天提醒 1 次，截止日当天提醒 2 次。
- **低优先级：**仅在截止日当天提醒 1 次。

### 2. 复习提醒规则（基于艾宾浩斯记忆曲线简化版）：

- 若开启复习提醒，系统在任务完成后，自动在 **1 天、3 天、7 天、15 天** 生成复习任务提醒。

### 3. 完成率计算规则：

- 日完成率 = (当日状态变更为“已完成”的任务数) / (当日状态为“待办”及以上的总任务数) \* 100%。

### 4. 打卡规则：

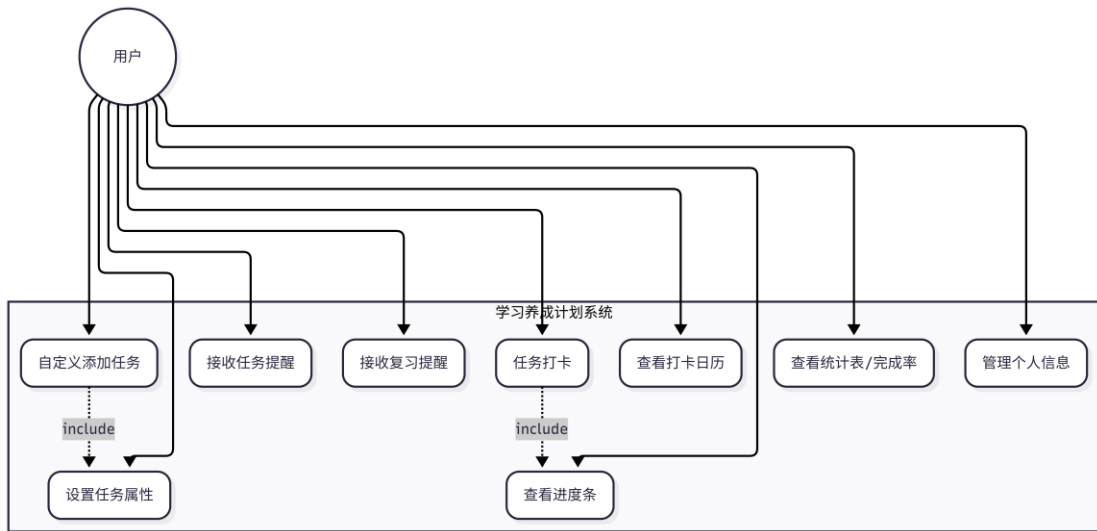
- 仅限当日手动打卡，逾期未打卡视为断签；若需补卡，需消耗系统虚拟积分或特权（可作为后续扩展需求）。

以上需求模型从用户视角、业务流转视角以及数据状态流转视角全面定义了“学习养成计划软件”的业务需求，可作为后续系统设计和代码生成的准确输入。

## 2. 需求分析建模

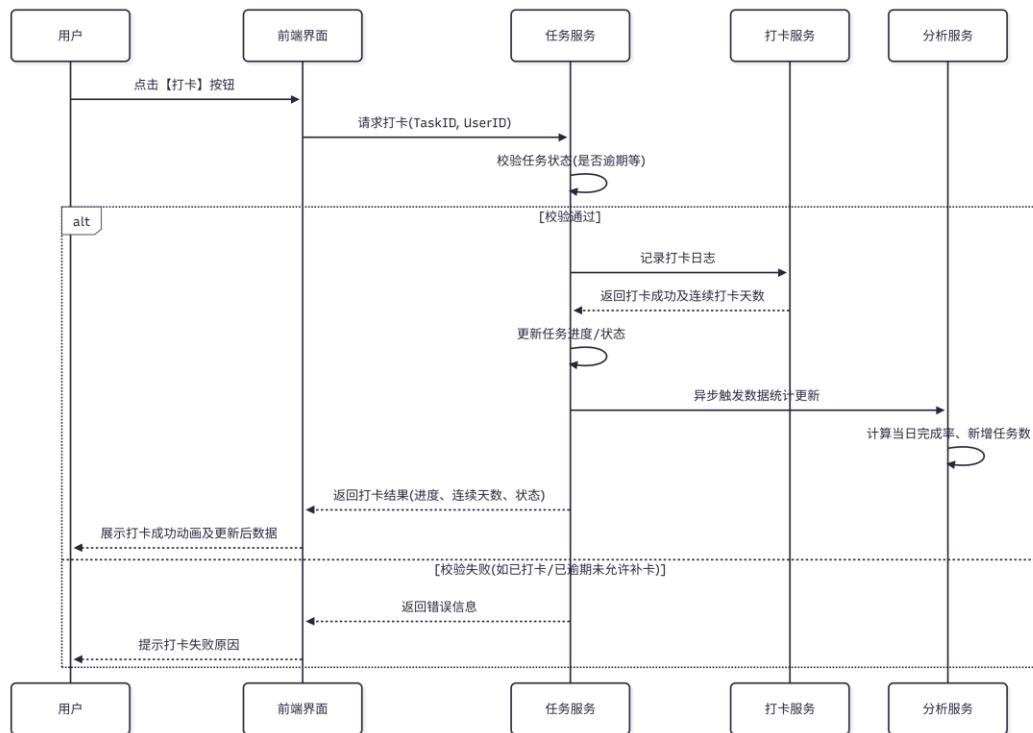
### 2.1 用例图

用例图展示了系统的参与者及其可执行的功能模块。



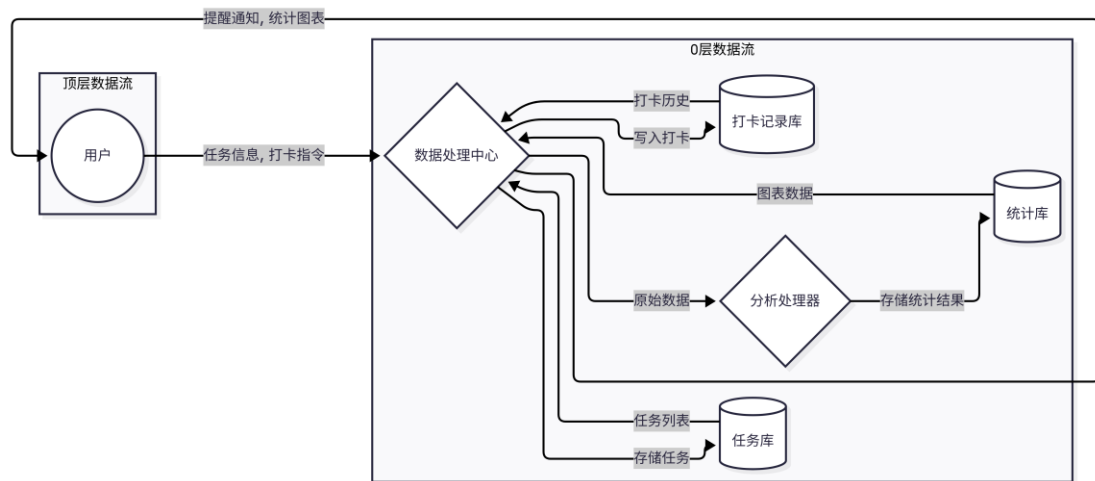
## 2 时序图

以核心业务\*\*“用户打卡及数据更新”\*\*为例，展示对象间的交互时序。



## 2.3 数据流图 (DFD)

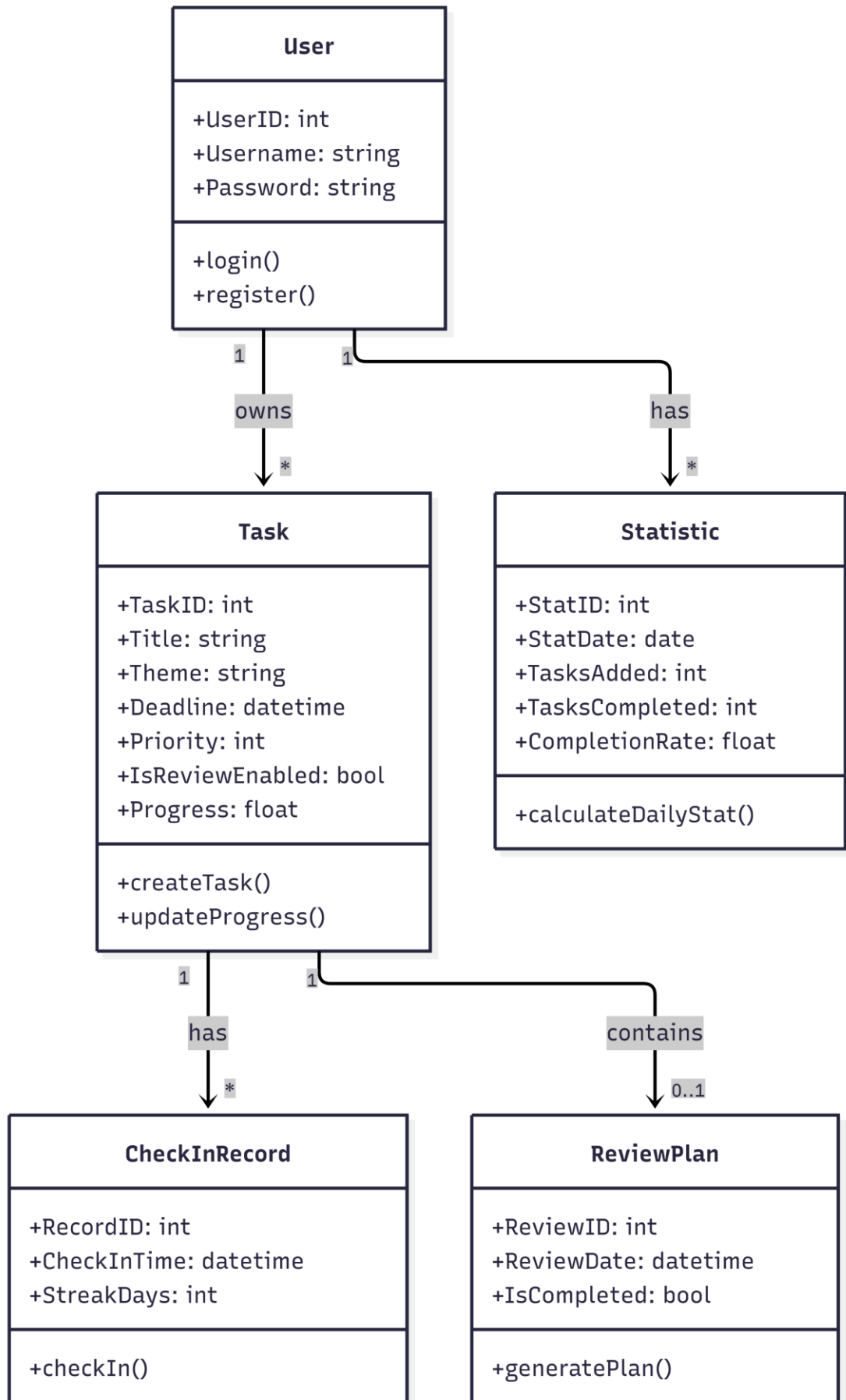
展示系统内数据的流动方向和处理过程。



## 2.4 类图

展示系统核心业务逻辑的实体类及其关系。



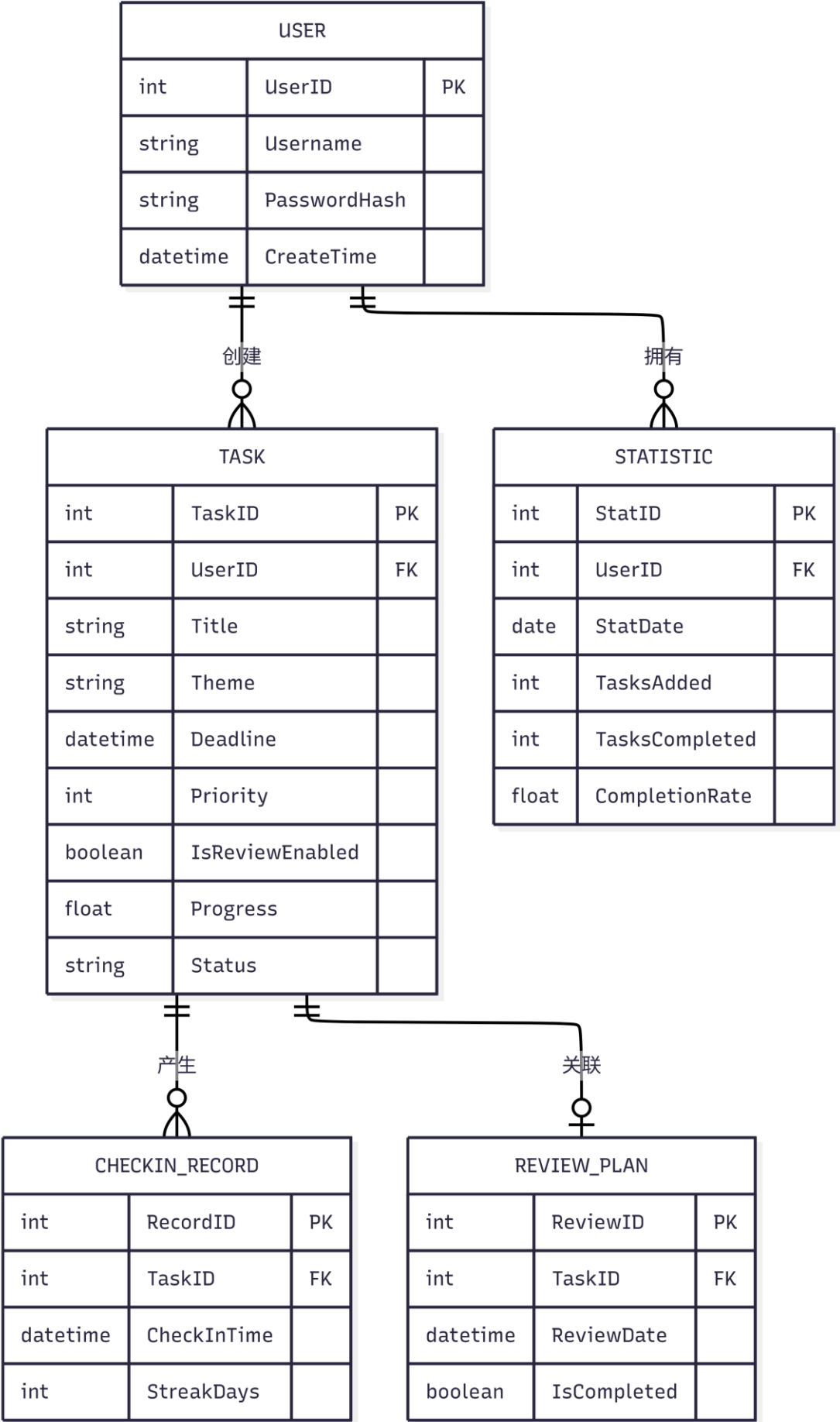


---

### 3. 数据建模：ER 图与数据字典

#### 3.1 实体关系图 (ER 图)

使用 Mermaid 绘制 ER 图，展示数据实体间的关系。



---

## 4. 非功能性需求说明

### 4.1 系统开发环境

- 前端/UI 开发：
  - 框架：WPF (Windows Presentation Foundation) 或 WinUI 3
  - 架构模式：MVVM (Model-View-ViewModel)
  - UI 组件库：HandyControl / MaterialDesignInXamlToolkit / WPF UI
  - 图表库：LiveCharts2 或 OxyChart (用于绘制进度条、统计表、打卡日历等可视化控件)
- 后端/逻辑层开发：
  - 语言：C# 10 / C# 11
  - 框架：.NET 6 / .NET 8
  - 数据访问：Entity Framework Core (SQLite Provider) 或 Dapper
- 数据库：
  - 本地关系型数据库：SQLite 3
- 开发工具：
  - IDE：Visual Studio 2022 / JetBrains Rider
  - 数据库管理：DB Browser for SQLite
  - 版本控制：Git

### 4.2 系统运行环境

- 操作系统要求：
  - 仅适配 Microsoft Windows 系统。
  - 最低支持：Windows 10 (版本 1809) 及以上。
  - 推荐支持：Windows 10 (21H2+) / Windows 11。
  - 系统架构：支持 x64 和 x86 架构，暂不支持 ARM64。
- 运行时依赖：
  - .NET Desktop Runtime 6.0.x / 8.0.x （可随安装包一同打包分发，避免用

户单独安装)。

- **硬件要求：**
  - 处理器：1 GHz 及以上处理器。
  - 内存：2 GB RAM 及以上。
  - 硬盘空间：预留至少 200 MB 可用空间（用于程序安装及本地数据库日志存储）。

#### 4.3 系统依赖项

- **系统级依赖：**
  - Windows Task Scheduler (任务计划程序)：用于在应用关闭时，依靠系统级定时任务触发本地提醒。
  - Windows Notification API (本地通知接口)：用于在系统右下角弹出 Toast 通知提醒。
- **NuGet 包依赖：**
  - Microsoft.EntityFrameworkCore.Sqlite：SQLite 数据库 ORM 支持。
  - LiveChartsCore 或 OxyPlot.Wpf：图表与可视化渲染。
  - Hardcodet.NotifyIcon.Wpf：用于支持系统托盘驻留（最小化到托盘）。
  - Microsoft.Toolkit.Uwp.Notifications：用于生成 Windows 原生的 Toast 通知弹窗。

#### 4.4 其他非功能性需求

- **性能需求：**
  - 客户端冷启动时间  $\leq 2$  秒。
  - 本地数据库查询响应时间  $\leq 100$  毫秒（数据量在 10 万条记录以内时）。
  - 统计图表与打卡日历渲染时间  $\leq 500$  毫秒。
  - 内存占用：空闲状态  $\leq 50$  MB，复杂交互状态  $\leq 150$  MB。
- **可靠性需求：**
  - 本地 SQLite 数据库需开启 WAL (Write-Ahead Logging) 模式，防止系统突然断电或崩溃导致数据库损坏。

- 支持本地数据的自动导出（备份）与导入（恢复）功能，备份格式为 .db 或 json。
- **安全性需求：**
  - 若支持本地密码锁功能，密码需使用 BCrypt 等强哈希算法存储在本地，禁止明文保存。
  - 数据库文件需存放在用户 AppData\Local 隐藏目录下，避免普通用户误删。
- **可用性需求：**
  - 支持开机自启动（可选配置），并默认最小化到系统托盘后台运行，不干扰用户日常使用。
  - 提醒弹窗需提供“稍后提醒（5 分钟/10 分钟）”和“标记完成”的快捷操作按钮，无需每次都打开主界面。